

程序设计实习大作业报告

一、程序功能介绍

本项目主要运用 python 的 flet 库等，构建了一个番茄钟类应用，并简要搭建了其可视化界面。项目的主要功能包括：看板操作、番茄钟使用、附件管理、到期提醒、统计视图、深色模式切换、导出完成情况以及今日目标设置 8 项。实操演示详见演示视频。

下面简要介绍 8 项基本功能：

1. 看板操作

添加列表：点击看板最右侧的“+ 添加列表”按钮。

添加卡片：点击列表底部的“+ 添加卡片”，输入标题即可。

编辑卡片：点击卡片打开详情对话框，可修改标题、备注、截止日期、提醒时间，管理附件和子任务。

移动卡片：

跨列表移动：鼠标按住卡片拖动到另一个列表后释放，弹出对话框选择“移动到其他列表”，然后选择目标列表。

同列表排序：拖拽卡片后选择“在当前列表排序”，使用置顶/上移/下移/置底按钮调整顺序。

删除卡片/列表：卡片右上角删除按钮（归档），列表右上角删除按钮（归档所有卡片）。

2. 番茄钟使用

切换到“番茄钟”标签页。

从下拉列表选择要绑定的任务卡片（也可不选择）。

点击“开始专注”，计时开始。

计时结束后自动进入短休息/长休息（根据设置），并弹出提醒。

可随时暂停/继续/重置。

点击右上角齿轮图标自定义专注/休息时长。

点击柱状图图标查看统计（周视图/月视图）。

3. 附件管理

在卡片详情对话框中，点击“上传文件”选择本地文件（支持任意格式）。

图片附件会出现“预览”按钮，点击可查看大图。

可下载或删除附件。

4. 到期提醒

程序每分钟自动检查卡片截止日期。

即将到期（默认提前 15 分钟）或已过期的卡片会在看板中高亮显示（橙色/红色背景），并在顶部弹出通知。

5. 统计视图

番茄钟界面右上角柱状图按钮打开统计对话框。

默认显示本周每日番茄数和专注分钟数。

点击“月”查看当月日历热力图（格子颜色深浅表示番茄数量）。

使用左右箭头切换不同周/月。

6. 深色模式

进入程序后，点击页面右上角“深色模式”按钮，即可切换为深色模式。

切换为深色模式后，点击页面右上角“浅色模式”按钮，即可切换回浅色模式。

7. 导出完成情况

进入程序后，按右上方下载图标，可以自动生成一个.txt 文件（存放在 reports 文件夹下），其中包含具体任务完成情况。

8. 今日目标设置

本功能支持设置今日专注时长目标。

在界面右上方设有设置目标和检查目标完成情况两个按钮。

设置目标按钮中，可以设置今日目标时长（单位为分钟，仅支持正整数），且支持随时修改。

检查目标完成情况按钮中，可以查看今日设置的目标、今日已经完成的专注时长，以及目标是否完成。

第二天再打开程序时，前一天设置的目标会自动重置。默认的目标为 25 分钟。

二、项目各模块与类设计细节

1. 看板管理模块设计

1.1 涉及的核心类与文件

看板管理模块由多个核心类协同完成。BoardView 类位于 views/board_view.py 文件中，负责看板主界面的渲染和用户交互响应，包括卡片的展示、拖拽移动、列表与卡片的增删操作。CardDetailDialog 类位于 views/card_detail_dialog.py 文件中，以弹窗形式提供卡片详情的编辑界面，包括标题修改、备注填写、截止日期与提醒时间设置，以及附件的上传、预览、下载和删除。BoardService 类位于 services/board_service.py 文件中，封装了看板、列表和卡片的所有业务逻辑，包括创建、更新、删除、移动、排序、归档与恢复等操作。JsonStorage 类位于 json_storage.py 文件中，负责所有数据以 JSON 格式持久化保存至本地磁盘，确保应用重启后数据不丢失。

1.2 列表与卡片的创建

系统启动时，JsonStorage 类自动创建包含待办、进行中、已完成三个标准列表的默认看板。看板主

界面采用横向滚动布局，每个列表以独立列呈现。列表顶部显示列表名称和卡片数量，底部提供添加卡片按钮。用户点击该按钮后，BoardView 类调用 BoardService 类的 add_card 方法完成卡片创建，新卡片随即出现在列表最下方。

1.3 卡片的拖拽移动与排序

卡片的移动支持两种模式：跨列表移动和同列表排序。用户通过鼠标拖拽卡片，BoardView 类中的手势处理方法捕获拖拽事件，在拖拽结束后弹出操作选择对话框。若用户选择移动到其他列表，BoardView 类调用 BoardService 类的 move_card 方法，将卡片从源列表移至目标列表的指定位置。若用户选择在当前列表排序，则弹出排序控制面板，包含置顶、上移、下移、置底四个按钮，BoardView 类调用 BoardService 类的 reorder_cards 方法重新调整卡片顺序。所有位置变化实时保存至存储层。

1.4 卡片详情的编辑与附件管理

卡片详情通过 CardDetailDialog 类进行编辑。用户点击任意卡片即弹出详情对话框，对话框内提供标题输入框、备注多行文本框、截止日期选择器和提醒时间选择器。用户修改任意字段后点击保存，CardDetailDialog 类调用 BoardService 类的 update_card 方法将更改写入存储。

附件管理集成在 CardDetailDialog 类中。用户点击上传文件按钮，通过系统文件选择器选取本地文件，CardDetailDialog 类将文件复制到 data/attachments 目录下，以通用唯一标识符重命名以避免文件名冲突，同时调用 BoardService 类的 add_attachment 方法将附件元数据写入卡片的 attachments 字段。对于图片格式的附件，界面中会出现预览按钮，点击后以弹窗形式展示大图。每个附件还提供下载和删除按钮，分别调用 BoardService 类的 download_attachment 方法和 remove_attachment 方法。

1.5 列表与卡片的删除

列表右上角提供删除按钮，点击后 BoardView 类调用 BoardService 类的 archive_list 方法，将该列表下的所有卡片标记为归档状态，随后删除列表本身。卡片右上角的删除按钮调用 BoardService 类的 archive_card 方法，将单张卡片标记为归档而非物理删除，保留数据以便日后查阅或恢复。

2. 番茄钟模块设计

2.1 涉及的核心类与文件

番茄钟模块由两个核心类协同完成。PomodoroView 类位于 views/pomodoro_view.py 文件中，负责计时器界面的构建与用户交互控制，包括计时器的启动、暂停、继续、重置，以及卡片选择下拉框、设置对话框和统计弹窗的调用。PomodoroService 类位于 services/pomodoro_service.py 文件中，封装了番茄钟会话管理的所有业务逻辑，包括会话的创建、暂停、恢复、停止，计时完成检查，设置管理，以及各类统计数据的查询与汇总。

2.2 计时器的启动与控制

计时器界面以大幅字体显示剩余时间，配合进度条可视化展示计时进度。用户从下拉列表中选择要绑定的任务卡片后，点击开始专注按钮，PomodoroView 类调用 PomodoroService 类的 start_session 方法启动计时会话。后台计时线程每秒更新剩余时间显示和进度条。计时结束时触发回调函数，根据当前状态弹出提醒对话框。系统自动按设定规则切换状态：完成一个专注时段后进入短休息，每完成四个专注时段后进入长休息。用户可随时点击暂停或继续按钮，PomodoroView 类相应调用 PomodoroService 类的 pause_session 方法和 resume_session 方法控制计时状态。

2.3 计时参数的设置

点击番茄钟界面右上角的齿轮图标打开设置对话框，用户可自定义专注时长、短休息时长、长休息时长和长休息间隔。设置通过 `PomodoroView` 类的 `open_settings` 方法调用 `PomodoroService` 类的 `update_settings` 方法写入本地存储，所有设置持久化保存在 `data/settings.json` 文件中。关闭设置对话框后，新参数立即生效，后续计时会话按照新设定运行。

2.4 任务绑定与快速创建

番茄钟界面的卡片选择下拉框通过 `PomodoroView` 类的 `_load_cards` 方法调用 `BoardService` 类的 `get_all_cards` 方法，获取所有看板中的未归档卡片列表供用户选择。用户选择一张卡片后，该卡片的唯一标识符会关联到当前计时会话。计时完成后，`PomodoroService` 类自动更新该卡片的番茄计数和总专注分钟数字段。

界面底部设有快速任务输入框，用户输入标题后按回车或点击添加按钮，`PomodoroView` 类调用 `BoardService` 类的 `add_card` 方法直接创建新卡片到待办列表，并自动刷新下拉列表，使用户无需切换到看板即可快速创建任务并开始专注。

2.5 计时完成的提醒与状态切换

计时结束后，系统根据当前会话类型执行不同的提醒与切换逻辑。若完成的是一个专注时段，系统弹出专注完成对话框，询问用户是否开始休息，用户确认后自动进入短休息计时。若完成的是休息时段，系统弹出休息结束对话框，提示用户开始下一轮专注。当连续完成四个专注时段后，系统自动进入长休息而非短休息。所有状态切换由 `PomodoroService` 类内部维护的状态机管理，确保计时流程符合番茄钟标准规则。

3. 统计与可视化模块设计

3.1 涉及的核心类与文件

统计模块由 `StatsDialog` 类和 `PomodoroService` 类协同完成。`StatsDialog` 类位于 `views/stats_dialog.py` 文件中，以弹窗形式构建统计界面的所有视觉元素，包括周视图列表卡片、月视图日历热力图、日期导航按钮和模式切换按钮。`PomodoroService` 类提供多个统计查询方法，包括 `get_daily_stats` 用于获取单日统计数据、`get_stats_by_range` 用于获取日期范围内的统计数据、`get_today_stats` 用于获取今日汇总数据。

目标设定功能由 `goal_setting.py` 文件中的三个独立函数实现，不依赖类封装。`get_today_goal` 函数读取 `data/daily_goal.json` 文件获取今日目标值，`set_today_goal` 函数将用户设定的目标值写入该文件，`get_pomodoro` 函数读取 `data/pomodoro.json` 中的每日统计字段获取今日实际专注时长。

3.2 统计弹窗的视图切换

点击番茄钟界面右上角的柱状图图标，`StatsDialog` 类构建统计弹窗并显示。弹窗默认展示本周视图，以列表卡片形式呈现本周每一天的数据，包括日期、番茄完成数、专注分钟数和完成任务数。用户点击月按钮切换至月视图，界面变为日历热力图。左右箭头按钮控制日期偏移，点击后 `StatsDialog` 类调用 `PomodoroService` 类的 `get_stats_by_range` 方法获取上一周或下一周、上月或下月的数据并刷新界面。

3.3 热力图的生成逻辑

月视图的核心是日历热力图。StatsDialog 类的 `_build_month_view` 方法获取当月每一天的统计数据后，为每个日期生成一个带颜色背景的容器。颜色透明度根据当日番茄数量从 0.1 到 1.0 线性映射，番茄数量越多颜色越深。无数据的日期显示为浅灰色背景。所有日期格子以网格形式整齐排列，每个格子上显示日期数字，形成直观的年度或月度专注分布图。

3.4 目标的设定与检查

界面右上角提供设置目标和检查完成情况两个按钮。点击设置目标按钮后弹出输入对话框，用户输入正整数作为今日专注时长目标，系统调用 `set_today_goal` 函数将目标值与当前日期一同写入 `data/daily_goal.json` 文件。点击检查完成情况按钮后，系统调用 `get_today_goal` 获取今日目标值，调用 `get_pomodoro` 获取今日已完成专注分钟数，比较两者后显示目标是否达成、还差多少分钟完成目标的提示信息。第二天重新打开程序时，日期判断逻辑使目标自动重置为默认的 25 分钟，确保每天独立设定。

4. 数据持久化与导出模块设计

4.1 涉及的核心类与文件

数据持久化由 `JsonStorage` 类统一管理，位于 `json_storage.py` 文件中。该类提供 `save` 和 `load` 方法的泛型实现，支持任意数据结构的读写操作。存储层维护三个核心 JSON 文件：`data/boards.json` 存储看板、列表、卡片的完整结构化数据，`data/pomodoro.json` 存储番茄钟会话记录和每日统计汇总数据，`data/settings.json` 存储用户自定义的计时器参数设置。

数据导出功能由 `data_export.py` 文件中的两个函数实现。`load_pomodoro_data` 函数加载 `data/pomodoro.json` 文件中的会话数据，`generate_report_from_pomodoro` 函数遍历所有会话记录，统计总专注时长、总番茄数、完成任务数，并按日期汇总每日详情，最终生成带时间戳的文本报告文件保存在 `reports` 目录下。

4.2 事务性存储机制

`JsonStorage` 类对所有写操作采用事务性保存策略。每次写入时，系统先将完整数据写入一个临时文件，写入成功后再将该临时文件重命名替换原目标文件。这种机制确保即使在写入过程中程序意外崩溃，原始数据文件也不会损坏，用户重新启动应用后仍可加载上一次完整保存的数据。

4.3 附件的存储与操作

附件物理文件存储在 `data/attachments` 目录下，每个附件以通用唯一标识符重命名，同时保留原始文件名作为元数据存入卡片的 `attachments` 字段。附件元数据包括原始文件名、存储路径、文件大小和上传时间。用户上传文件时，`CardDetailDialog` 类完成文件复制和元数据写入；用户预览图片时，系统根据存储路径加载图片文件并显示；用户下载附件时，系统按原始文件名提供文件下载；用户删除附件时，系统同时删除物理文件和元数据记录。

4.4 到期提醒机制

到期提醒功能由 `BoardService` 类的 `check_overdue_cards` 方法和 `check_today_due_cards` 方法实现。系统启动后启动一个后台定时器，每隔一分钟调用一次上述方法。检查逻辑遍历所有未归档卡片，计算当前时间与卡片截止日期的差值。若卡片距离截止日期不足十五分钟但尚未过期，卡片在看板中显示橙色背景高亮，并在页面顶部弹出即将到期的通知。若卡片已过截止日期，卡片显示红色背景高亮，弹出已过期的通知。高亮样式的判断和渲染在 `BoardView` 类的 `_create_card_widget` 方法中根据卡片

状态标志动态完成。

4.5 数据报告的生成

用户点击界面右上角的下载图标后,系统调用 `data_export.py` 中的 `generate_report_from_pomodoro` 函数。该函数首先调用 `load_pomodoro_data` 加载所有历史会话记录,然后遍历每条记录累加总专注时长和总番茄数,同时按卡片唯一标识符去重统计完成任务数。随后按日期分组汇总每日的专注时长、番茄数和完成任务数。报告以纯文本格式生成,文件名包含生成时间戳,保存在 `reports` 文件夹下。生成成功后系统弹出提示对话框告知用户文件保存路径,生成失败时显示具体错误信息。

5. 主题切换与界面适配设计

5.1 涉及的核心类与文件

主题管理由 `ThemeManager` 类负责,位于 `dark_theme.py` 文件中。该类管理 Flet 应用的主题切换功能,提供主题初始化、切换逻辑和切换按钮生成三个核心方法。`init` 方法接收页面对象并保存,设置初始主题模式为浅色,调用 `_setup_themes` 方法配置浅色和深色两套主题配色方案。`toggle` 方法检测当前主题模式并切换到另一模式,同时更新按钮文字。`get_button` 方法返回一个用于触发主题切换的按钮控件。

5.2 主题配色方案

`_setup_themes` 方法分别设置 `page.theme` 和 `page.dark_theme` 两个属性,两套主题的配色种子均指定为蓝色,确保浅色模式和深色模式下的主色调保持一致。浅色模式采用明亮背景和深色文字,深色模式采用深色背景和浅色文字,所有控件颜色由 Flet 框架根据主题种子自动派生,保证界面各元素在两种模式下均有良好的对比度和可读性。

5.3 主题切换流程

用户点击主题切换按钮后, `toggle` 方法执行切换逻辑。若当前为浅色模式,将 `page.theme_mode` 设置为深色,按钮文字改为浅色模式。若当前为深色模式,将 `page.theme_mode` 设置为浅色,按钮文字改为深色模式。主题切换后,如果页面对象存在 `board_view` 属性, `toggle` 方法调用该属性的 `refresh_by_theme` 方法强制刷新看板中的所有卡片样式,确保卡片的高亮颜色和背景色立即响应主题变化。最后调用 `page.update` 方法将界面更新生效。

5.4 按钮的集成位置

`get_button` 方法返回的切换按钮被放置在主界面的右上角,与目标设定按钮、数据导出按钮并排排列。按钮初始文字为深色模式,表示点击后将切换至深色主题。用户点击后按钮文字同步更新,清晰指示下一次点击将切换到的模式,避免用户混淆当前状态与操作结果。

三、小组成员分工情况

1. 分工原则

- 根据项目模块划分职责,确保每个核心功能有人负责。
- 兼顾前后端协作,保证接口定义清晰。
- 设立项目负责人统筹进度和质量。

2. 具体分工表

| 成员姓名 | 负责模块/功能 |

|-----|-----|

|林宥安| 整体架构设计、Flet GUI 框架搭建、番茄钟视图 (PomodoroView)、FastAPI 服务、JSON 存储层 (JsonStorage)、API 路由

|王健宁|业务逻辑 (BoardService、PomodoroService)、统计弹窗 (StatsDialog)、深色模式、数据导出、每日目标、计时器线程、功能测试、实验报告撰写

|王梓鑫|看板视图 (BoardView)、卡片详情 (CardDetail)、拖拽手势实现、附件上传/预览、到期提醒通知、用户文档 (README)、功能测试、实验报告撰写

3. 详细职责描述

林宥安

- 架构设计：确定三层架构（视图-服务-存储），设计各层接口及数据流向。
- GUI 基础框架：搭建 Flet 主窗口、标签页布局、全局状态管理
- 数据模型：定义 `models/board.py` 和 `models/pomodoro.py` 中的数据类 (Board, Card, PomodoroSession, DailyStats)。
- 存储层：实现 `JsonStorage` 的读写方法，确保数据持久化正确 (`boards.json`, `pomodoro.json`, `settings.json`)。
- API 接口：使用 FastAPI 定义 REST API (`server.py`)，支持 C++ Qt 客户端调用。
- 番茄钟视图：构建 `PomodoroView` 的计时界面（显示、进度条、开始/暂停/重置按钮）、卡片下拉选择、任务详情面板。
- 演示准备：准备项目演示材料（截图、操作视频）。

王健宁

- 服务层：编写 `BoardService`（看板 CRUD、移动、排序、搜索、标签）和 `PomodoroService`（会话管理、统计查询）。
- 统计弹窗：开发 `StatsDialog` 显示周/月统计数据，支持日期导航。
- 深色模式：开发 `dark_theme.py` 切换逻辑，并适配看板卡片背景色。
- 跨服务协调：确保前端调用 `board_service` 和 `pomodoro_service` 方法返回统一格式数据。
- UI 适配：确保各控件在深色/浅色模式下显示正常。
- 数据导出与目标：实现 `data_export.py` 生成报告、`goal_setting.py` 设置今日目标。
- 计时逻辑：编写 `timer_loop` 后台线程，实现计时更新、自动切换状态（专注→休息）、结束时弹出提醒。

- 功能测试：编写测试用例，覆盖看板操作、番茄钟计时、统计显示、附件上传等核心流程。
- 实验报告：整理项目背景、设计、实现、测试、总结等章节，形成最终实验报告。

王梓鑫

- 看板模块：实现 BoardView 的列表渲染、卡片增删改、拖拽菜单（移动/排序）、到期提醒检查线程。
- 卡片详情对话框：实现 CardDetailDialog 的字段编辑、附件管理（上传/下载/删除/图片预览）。
- 拖拽手势：在 BoardView 中用 GestureDetector 替代原生 Draggable，实现卡片拖拽后的移动/排序菜单。
- 附件交互：集成 ft.FilePicker 实现文件选择，处理图片预览、下载保存。
- 功能测试：编写测试用例，覆盖看板操作、番茄钟计时、统计显示、附件上传等核心流程。
- 用户文档：编写 README.md，包含安装步骤、使用指南、API 说明。
- 实验报告：整理项目背景、设计、实现、测试、总结等章节，形成最终实验报告。

四、项目总结与反思

4.1 所遇到的问题

问题 1: Flet 原生 Draggable/DragTarget 拖拽不触发

现象: Draggable 的 on_drag_start 触发，但 DragTarget 的 on_accept 不响应。

分析: Flet 0.25.1 的拖拽机制在 Windows 部分系统上存在兼容性问题，组名匹配和数据传递可能被底层 Flutter 事件拦截。

解决方案: 放弃原生 Draggable/DragTarget，改用 GestureDetector 手动实现拖拽：

- on_pan_start: 记录卡片 ID 和源列表 ID
- on_pan_update: 更新浮动视觉反馈
- on_pan_end: 弹出操作选择对话框（移动/排序）
- 移动调用 BoardService.move_card()，排序调用 BoardService.reorder_cards()

问题 2: 统计视图集成导致黑屏

现象: 将统计视图作为 ft.Tabs 的第三个标签页后，整个程序启动后黑屏，无任何错误输出。

分析: Flet 在构建多标签页时，如果某个标签页内容在 build 阶段调用了 self.update() 或使用了 expand=True 嵌套，可能导致布局引擎陷入无限循环或控件未绑定页面就尝试更新。

解决方案:

1. 将统计功能改为弹窗 (StatsDialog 继承 ft.AlertDialog), 避免标签页布局冲突。
2. 在 did_mount() 中调用 _load_data(), 确保控件已添加到页面后再更新。
3. 使用 ft.ListView(expand=1) 替代 ft.Column 的滚动, 避免 expand 嵌套问题。

问题 3: 卡片移动后源列表卡片不消失

现象: 控制台日志显示移动成功 (源列表卡片数减少、目标列表增加), 但 UI 上源列表的卡片仍然可见。

分析: 前端 _load_board() 重新从后端获取数据后, update_ui() 调用了 self.board_row.controls.clear() 并重新构建, 但 Flet 的 Column 内嵌套的 Container 可能保留了旧引用。

解决方案: 在 update_card 方法中, 当检测到 list_id 变更时, 先从原列表中移除卡片, 再添加到新列表, 确保数据一致性。同时在前端 update_ui() 中完全重建控件树, 而非尝试增量更新。

4.2 项目反思

(1) 做得好的地方

1. 分层架构清晰: 视图、服务、存储、模型四层分离, 代码可维护性高。
2. 异常处理逐步完善: 从最初的黑屏无反馈, 到后来每个关键模块都有 try-except 和日志输出。
3. 用户界面友好: 提供深色模式、每日目标、数据导出等增强体验的功能。

(2) 可以改进的地方

1. 测试驱动开发不足: 项目开发过程中缺乏单元测试和集成测试, 导致回归问题频发。后续应先编写核心服务层的测试用例。
2. 统计功能降级: 统计视图最终以弹窗形式实现而非标签页, 是技术妥协, 未来应研究如何稳定地在标签页中嵌入复杂组件。
3. 性能优化: 看板每次刷新都完全重建所有列表和卡片, 当卡片数量超过 100 张时会出现明显卡顿。

(3) 技术选型反思

Flet 的优点:

- Python 语法简洁, 开发速度快
- 跨平台 (Windows/Mac/Linux)
- 社区活跃, 文档丰富

Flet 的局限：

- 版本迭代快，API 变动频繁
- 部分高级功能（如精确拖拽）需要绕道实现
- 大型应用性能不如原生 Qt

4.3 个人成长与收获

1. 框架驾驭能力：通过手动实现拖拽、解决黑屏问题，加深了对 Flet 布局机制和事件循环的理解。
2. 调试方法论：掌握了“二分注释法”和“最小复现法”，在无错误日志时也能快速定位问题。
3. 面向对象设计：通过分层架构的设计与重构，理解了依赖注入和单一职责原则的实际应用。
4. 时间管理意识：在开发过程中，番茄钟功能不仅是为用户设计的，也帮助自己保持专注——这正是“做工具的人首先使用工具”的体现。

4.4 总结

Trello Pomodoro 项目从设计到实现，经历了需求分析、架构设计、编码实现、问题修复、功能完善等完整流程。过程中遇到的每个问题都成为了学习机会，最终产出了一款功能完整、运行稳定的桌面应用。项目虽小，但五脏俱全，为后续更复杂的项目积累了宝贵的经验。